

Python pour le trading — Document 5/8

La manipulation de données

Fil conducteur. Toujours le gestionnaire d'ordres de trading. Ce document est ton terrain : ton expérience en apprentissage automatique te donne déjà une bonne base ici. Manipuler efficacement des données de marché — prix, volumes, séries temporelles — est au cœur du métier. On y voit NumPy (le calcul rapide), Pandas (les tableaux de données) et la gestion délicate du temps.

Comment lire ce document. Chaque section présente d'abord l'idée en mots simples, puis un exemple commenté, puis une explication. Le texte entre les blocs de code porte l'essentiel : ne le saute pas.

Table des matières

1. NumPy : le calcul vectorisé

- 1.1 La vectorisation
- 1.2 Vectorisation, code natif et GIL
- 1.3 Opérations utiles
- 1.4 Création et inspection d'un array

2. Pandas : les DataFrames

- 2.1 Créer et explorer un DataFrame
- 2.2 Exemples utiles au quotidien
- 2.3 Manipulations essentielles de nettoyage
- 2.4 Filtrer et grouper
- 2.5 Les séries temporelles
- 2.6 Les dates et les formats de date

3. La gestion du temps et des fuseaux horaires

- 3.1 Dates naïves et dates avisées
- 3.2 Convertir entre fuseaux
- 3.3 Construire et formater des dates
- 3.4 Opérations de temps essentielles

4. Comprendre les paramètres clés

- 4.1 Les paramètres positionnels et nommés
- 4.2 Les paramètres essentiels à connaître

Récapitulatif

Questions de vérification

1. NumPy : le calcul vectorisé

NumPy fournit l'**array**, un tableau de nombres bien plus rapide qu'une liste Python pour le calcul. Son secret est la **vectorisation** : appliquer une opération à tout le tableau d'un coup, au lieu de boucler élément par élément. C'est la base de toute la pile data de Python.

1.1 La vectorisation

```
import numpy as np
```

```
prix = np.array([5230.0, 5231.5, 5232.0, 5233.25])
```

```
# SANS vectorisation (boucle Python, lent)
```

```
resultat = []
```

```
for p in prix:
```

```
    resultat.append(p * 1.01)
```

```
# AVEC vectorisation (une seule opération, rapide)
```

```
resultat = prix * 1.01      # applique * 1.01 à TOUT le tableau d'un coup
```

```
# -> array([5282.3, 5283.815, 5284.32, 5285.5825])
```

Avec NumPy, `prix * 1.01` multiplie chaque élément sans qu'on écrive de boucle. Ce n'est pas qu'une question de concision : sous le capot, l'opération est exécutée en code C optimisé, donc bien plus vite qu'une boucle Python. Sur des millions de prix, l'écart de vitesse est considérable. La règle d'or de NumPy : éviter les boucles Python, penser en opérations sur des tableaux entiers.

1.2 Vectorisation, code natif et GIL : le lien avec le document 4

La vectorisation NumPy délègue le calcul à du **code natif** (C/Fortran, via des bibliothèques comme BLAS, MKL ou OpenBLAS). Cela apporte **deux** gains distincts qu'il faut bien séparer :

1. On évite l'interpréteur Python. Même sur un seul cœur, le code C est intrinsèquement plus rapide qu'une boucle Python, car il ne re-vérifie pas le type à chaque élément. C'est la source principale du gain (souvent un facteur 10 à 100).

2. Le code natif relâche le GIL. Pendant qu'il s'exécute, ce code C libère le verrou global (le GIL, vu au document 4). Cela permet, pour les opérations lourdes (typiquement l'algèbre matricielle comme $A @ B$), de **paralléliser sur plusieurs cœurs** à l'intérieur de la bibliothèque. Une boucle Python pure, elle, garderait le GIL et resterait mono-cœur.

***Pourquoi ça compte.** Toutes les opérations NumPy ne sont pas multi-cœurs. Les opérations simples élément par élément (`prix * 1.01`) sont surtout accélérées par le point 1 (code C, mono-cœur). Les grosses opérations d'algèbre linéaire profitent en plus du point 2 (multi-cœurs via BLAS). Dans les deux cas, on est plus rapide qu'une boucle Python ; mais « libérer le GIL » et « être plus rapide » ne sont pas tout à fait la même chose.*

Pourquoi ça compte. Calculer des rendements, des moyennes mobiles ou des indicateurs sur des millions de barres en boucle Python serait trop lent. La vectorisation NumPy rend ces calculs quasi instantanés, ce qui est indispensable pour la recherche quantitative.

1.3 Opérations utiles

```
prix = np.array([5230.0, 5231.5, 5232.0, 5233.25, 5234.50])
```

```
prix.mean()          # moyenne -> 5232.25
```

```
prix.std()           # écart-type (volatilité)
```

```
prix.max() - prix.min() # amplitude (range)
```

```
np.diff(prix)         # différences successives -> rendements bruts
```

```
prix[prix > 5232]     # filtre booléen : garde les prix > 5232
```

NumPy offre des opérations statistiques directes (mean, std...) et le **filtrage booléen** : `prix[prix > 5232]` renvoie tous les prix supérieurs à 5232, sans boucle ni condition explicite. `np.diff` calcule les différences successives — la base du calcul des rendements. Ces opérations vectorisées sont les briques de toute analyse de marché.

Calculer des rendements (exemple concret) :

```
# Rendement simple entre chaque prix et le précédent
rendements = np.diff(prix) / prix[:-1] # (p[t] - p[t-1]) / p[t-1]
rendements.mean() # rendement moyen
rendements.std() # volatilité (écart-type des rendements)
```

On combine `np.diff` (les écarts successifs) et `prix[:-1]` (tous les prix sauf le dernier) pour obtenir les rendements relatifs, en une ligne vectorisée. La moyenne et l'écart-type de ces rendements donnent directement le rendement moyen et la volatilité.

Les opérations NumPy importantes en un coup d'œil :

Opération	Ce qu'elle fait
<code>np.array(liste)</code>	Crée un array NumPy à partir d'une liste Python.
<code>a.mean()</code>	Moyenne des éléments.
<code>a.std()</code>	Écart-type (dispersion ; en finance, la volatilité).
<code>a.sum()</code>	Somme de tous les éléments.
<code>a.min()</code> / <code>a.max()</code>	Plus petite / plus grande valeur.
<code>a.max() - a.min()</code>	Amplitude (l'étendue entre le min et le max).
<code>np.diff(a)</code>	Différences successives <code>a[i+1] - a[i]</code> ; base des rendements.
<code>a[a > seuil]</code>	Filtrage booléen : ne garde que les éléments vérifiant la condition.
<code>a * 1.01</code>	Opération vectorisée : applique le calcul à TOUT le tableau.
<code>a @ b</code>	Produit matriciel ; opération lourde qui peut utiliser plusieurs cœurs.
<code>np.cumsum(a)</code>	Somme cumulée (utile pour un PnL cumulé).
<code>a.reshape(1, c)</code>	Réorganise le tableau en 1 lignes et c colonnes.

1.4 Création et inspection d'un array

Au-delà de `np.array`, quelques fonctions servent à créer rapidement des tableaux (pour initialiser, tester, ou générer des séries) et à inspecter leur forme.

```
np.zeros(5) # array de 5 zéros : [0. 0. 0. 0. 0.]
np.ones(3) # array de 3 uns
np.arange(0, 10, 2) # de 0 à 10 par pas de 2 : [0 2 4 6 8]
np.linspace(0, 1, 5) # 5 valeurs régulières de 0 à 1
np.random.randn(1000) # 1000 tirages aléatoires (loi normale)
```

```
a = np.array([[1, 2, 3], [4, 5, 6]])
a.shape # (2, 3) : 2 lignes, 3 colonnes
a.dtype # le type des éléments (ex. int64, float64)
a.size # nombre total d'éléments : 6
```

`zeros`, `ones`, `arange` et `linspace` créent des tableaux sans partir d'une liste — pratique pour initialiser un buffer de prix ou générer un axe temporel. `shape`, `dtype` et `size` renseignent sur la structure d'un

array, le premier réflexe quand on découvre des données. `np.random.randn` génère des données aléatoires, utile pour tester une stratégie avant d'avoir de vraies données.

Pourquoi ça compte. Connaître `shape` et `dtype` d'un array (ou plus tard d'un `DataFrame`) est le tout premier geste d'inspection : il révèle immédiatement la taille des données et d'éventuels types inattendus (des prix stockés en entier, par exemple).

2. Pandas : les DataFrames

Pandas bâtit sur NumPy et fournit le **DataFrame** : un tableau à colonnes nommées, comme une feuille de calcul ou une table de base de données. C'est l'outil de référence pour manipuler des données de marché structurées (date, symbole, prix, volume...).

2.1 Créer et explorer un DataFrame

```
import pandas as pd

df = pd.DataFrame({
    "symbole": ["ES", "NQ", "ES", "CL"],
    "prix":     [5234.50, 18450.0, 5235.0, 78.20],
    "volume":   [1200, 800, 1500, 600],
})

df.head()          # aperçu des premières lignes
df["prix"]          # une colonne
df["prix"].mean()   # moyenne de la colonne prix
df.describe()       # statistiques résumées de toutes les colonnes
```

Un `DataFrame` se construit ici à partir d'un dictionnaire où chaque clé devient une colonne. On accède à une colonne par son nom (`df["prix"]`), et toutes les opérations NumPy s'y appliquent (`.mean()`). `describe()` donne d'un coup les statistiques de chaque colonne. C'est exactement le format dans lequel on charge des historiques de marché.

2.2 Exemples utiles au quotidien

Voici les manipulations les plus fréquentes en pratique, dont le chargement depuis un fichier (le point de départ de presque toute analyse).

Charger un fichier dans un DataFrame :

```
# Depuis un CSV (un relevé de courtier, un historique exporté)
df = pd.read_csv("historique_es.csv")

# Depuis un fichier Parquet (format compact pour gros volumes, voir doc 6)
df = pd.read_parquet("historique_es.parquet")

# Préciser qu'une colonne contient des dates, et en faire l'index
df = pd.read_csv("ticks.csv", parse_dates=["date"], index_col="date")
```

Sélectionner, ajouter, transformer :

```
df["prix"]          # une colonne (Series)
df[["symbole", "prix"]] # plusieurs colonnes (sous-DataFrame)
df.iloc[0]          # la première ligne (par position)
df.loc[df["symbole"] == "ES"] # les lignes ES (par condition)
```

```
# Ajouter une colonne calculee (vectorise, sans boucle)
df["valeur"] = df["quantite"] * df["prix"]

# Statistiques rapides
df["prix"].describe()          # count, mean, std, min, quartiles, max
df["symbole"].value_counts()   # combien de lignes par symbole

# Connaitre la taille du DataFrame
len(df)                        # nombre de lignes
df.shape                       # (nombre de lignes, nombre de colonnes)
```

Pourquoi ça compte. *read_csv* est souvent la toute première ligne d'un script d'analyse : on charge un relevé ou un historique, puis on enchaîne filtres, colonnes calculées et agrégats. Pour de gros volumes, *read_parquet* est bien plus rapide (voir document 6).

2.3 Manipulations essentielles de nettoyage et de préparation

Avant d'analyser des données de marché, il faut presque toujours les **nettoyer et les préparer** : harmoniser les noms de colonnes, traiter les valeurs manquantes, corriger les types, normaliser les échelles, fusionner des sources. Ces opérations, familières en apprentissage automatique, sont au cœur du travail quotidien sur les données.

Renommer des colonnes

```
df = df.rename(columns={"px": "prix", "qty": "quantite"}) # par dictionnaire
df.columns = ["symbole", "prix", "quantite", "date"]      # tous d'un coup
df.columns = df.columns.str.strip().str.lower()           # normaliser les noms
```

Le `rename` est essentiel quand on réconcilie deux sources aux conventions différentes (un courtier nomme « px », un autre « price ») : on harmonise avant de comparer.

Changer le type d'une colonne

Après un chargement, une colonne peut avoir le mauvais type : un nombre lu comme texte se trie et se calcule à tort. La méthode principale est `astype`.

```
df["quantite"] = df["quantite"].astype(int)      # vers entier
df["prix"] = df["prix"].astype(float)           # vers nombre a virgule
df = df.astype({"quantite": int, "prix": float}) # plusieurs d'un coup
df.dtypes                                         # verifier les types
```

```
# Nombres mal formes : NaN au lieu de planter
df["prix"] = pd.to_numeric(df["prix"], errors="coerce")
```

```
# Virgule decimale (format europeen "5234,50")
df["prix"] = df["prix"].str.replace(",", ".").astype(float)
```

`astype` convertit vers le type voulu ; `pd.to_numeric` avec `errors="coerce"` met NaN sur les valeurs illisibles au lieu de planter ; et pour un séparateur décimal européen, on remplace la virgule par un point avant de convertir.

Gérer les valeurs manquantes (NaN)

```
df.isna().sum()          # compter les valeurs manquantes par colonne
df = df.dropna()         # supprimer les lignes incompletes
df = df.fillna(0)        # remplacer les manquantes par une valeur
df["prix"] = df["prix"].ffill() # propager la derniere valeur connue
```

Les données de marché ont souvent des trous. `dropna` supprime les lignes incomplètes ; `fillna` les comble ; `ffill` propage la dernière valeur connue — utile pour un prix inchangé. Le choix dépend du contexte.

Tester les valeurs de certaines colonnes

```
# Tester une condition sur 4 colonnes parmi 8, ligne par ligne
cols = ["prix", "quantite", "volume", "ouverture"]
masque = (df[cols] > 0).all(axis=1)      # True si les 4 valeurs > 0 sur la ligne
valides = df[masque]                    # lignes conformes
fautives = df[~masque]                   # lignes qui échouent
```

```
df[df[cols].isna().any(axis=1)]          # lignes avec un trou dans ces colonnes
(df[cols] <= 0).sum()                     # nb de valeurs <= 0 par colonne
```

On sélectionne le sous-ensemble `df[cols]`, on applique la condition à tout le bloc, et `.all(axis=1)` vérifie **ligne par ligne** que les colonnes ciblées la satisfont (`axis=1` = horizontal). `.any(axis=1)` teste « au moins une » au lieu de « toutes ».

Normaliser et mettre à l'échelle

```
# Normalisation min-max : ramener une colonne dans [0, 1]
df["prix_norm"] = (df["prix"] - df["prix"].min()) / (df["prix"].max() -
df["prix"].min())
```

```
# Standardisation (score z) : moyenne 0, écart-type 1
df["prix_z"] = (df["prix"] - df["prix"].mean()) / df["prix"].std()
```

```
# Rendements (variation relative d'une ligne à l'autre)
df["rendement"] = df["prix"].pct_change()
```

La **normalisation min-max** ramène les valeurs entre 0 et 1 ; la **standardisation** (score z) les centre sur une moyenne de 0. Étapes classiques avant d'alimenter un modèle. `pct_change` calcule les rendements, plus parlants que les prix bruts pour comparer des actifs d'échelles différentes.

Trier, dédupliquer, réindexer

```
df = df.sort_values("date")              # trier par date (croissant par défaut)
df = df.sort_values("prix", ascending=False) # décroissant
df = df.drop_duplicates()                 # retirer les lignes en double
df = df.reset_index(drop=True)            # renuméroter l'index proprement
df = df.set_index("date")                 # faire d'une colonne l'index
```

Trier par date est indispensable pour une série temporelle (`ascending=False` pour l'ordre décroissant). `set_index("date")` transforme la colonne de dates en index, ce qui débloque ensuite `resample` et `rolling`.

Trier les valeurs à l'intérieur d'une ligne

```
# Trier les VALEURS d'une ligne, sur un sous-ensemble de colonnes seulement
cols = df.columns[1:4]                    # colonnes d'indices 1, 2, 3
df[cols] = df[cols].apply(
    lambda ligne: sorted(ligne),           # sorted(..., reverse=True) -> décroissant
    axis=1, result_type="expand",
)
```

`axis=1` fait travailler `apply` **ligne par ligne** ; seules les colonnes ciblées sont réécrites, l'index et les autres colonnes restent intacts. À noter : les valeurs triées restent sous les mêmes noms de colonnes (la plus petite dans la première colonne de `cols`).

Fusionner et concaténer des sources

```
# Empiler deux DataFrames de meme structure (ex. deux journees)
df = pd.concat([df_lundi, df_mardi])
```

```
# Joindre deux sources sur une cle commune (positions + releve courtier)
fusion = pd.merge(positions, releve, on="symbole", how="inner")
```

concat empile des DataFrames ; merge joint deux sources sur une colonne commune — exactement l'opération d'une réconciliation : rapprocher positions internes et relevé du courtier, puis comparer.

Appliquer une transformation

```
df["cote"] = df["cote"].map({"B": "achat", "S": "vente"}) # correspondance
df["categorie"] = df["volume"].apply(lambda v: "gros" if v > 1000 else "petit")
```

map remplace des valeurs selon une correspondance ; apply applique une fonction quelconque, mais reste plus lent qu'une opération vectorisée : à réserver au non-vectorisable.

Pourquoi ça compte. Le nettoyage et la préparation occupent souvent la majeure partie du temps d'un travail sur données. Renommer, corriger les types, combler les trous, normaliser et fusionner proprement, c'est ce qui sépare une analyse fiable d'un résultat faussé par des données mal alignées — un point critique pour la réconciliation et le reporting.

Les manipulations essentielles en un coup d'œil :

Opération	Ce qu'elle fait
<code>df.rename(columns={...})</code>	Renomme certaines colonnes (ancien -> nouveau).
<code>df.columns = [...]</code>	Remplace tous les noms de colonnes.
<code>df["c"].astype(type)</code>	Change le type d'une colonne (int, float, str, bool).
<code>pd.to_numeric(col, errors="coerce")</code>	Convertit en nombre ; NaN sur les valeurs illisibles.
<code>df.isna().sum()</code>	Compte les valeurs manquantes par colonne.
<code>df.dropna()</code>	Supprime les lignes contenant des valeurs manquantes.
<code>df.fillna(v) / .ffill()</code>	Comble les manquantes / propage la precedente.
<code>(df[cols] > 0).all(axis=1)</code>	Teste une condition sur plusieurs colonnes, ligne par ligne.
<code>(x-x.min())/(x.max()-x.min())</code>	Normalisation min-max dans [0, 1].
<code>(x - x.mean()) / x.std()</code>	Standardisation (score z).
<code>x.pct_change()</code>	Rendements : variation relative d'une ligne a l'autre.
<code>df.sort_values("col")</code>	Trie les lignes (croissant ; ascending=False pour decroissant).
<code>df.drop_duplicates()</code>	Retire les lignes en double.
<code>df.set_index("col")</code>	Fait d'une colonne l'index (utile pour resample).
<code>pd.concat([a, b])</code>	Empile des DataFrames de meme structure.
<code>pd.merge(a, b, on="cle")</code>	Joint deux sources sur une cle commune.
<code>df["c"].map({...})</code>	Remplace des valeurs selon une correspondance.

2.4 Filtrer et grouper

```
# Filtrer : tous les ordres sur ES
```

```
df[df["symbole"] == "ES"]

# Grouper : volume total par symbole
df.groupby("symbole")["volume"].sum()
# -> ES: 2700, NQ: 800, CL: 600

# Trier par prix décroissant
df.sort_values("prix", ascending=False)
```

Le **filtrage** `df[df["symbole"] == "ES"]` garde les lignes correspondant à une condition. Le `groupby` regroupe les lignes par valeur (ici par symbole) puis applique un calcul (la somme des volumes) — l'équivalent d'un tableau croisé dynamique. Ces deux opérations sont le pain quotidien de l'analyse de données de marché.

Pourquoi ça compte. *Réconcilier des positions, agréger des volumes par symbole, calculer un PnL par stratégie : tout cela se fait en quelques lignes de Pandas. C'est l'outil central pour le reporting et l'analyse mentionnés dans l'offre Tower.*

2.5 Les séries temporelles

Pandas excelle sur les **séries temporelles**, indispensables en trading. On peut indexer par date et rééchantillonner (changer la fréquence).

```
# Index temporel : chaque ligne est datee
df = pd.DataFrame({"prix": [5230, 5231, 5232, 5233]},
                  index=pd.to_datetime([
                      "2026-06-14 09:30", "2026-06-14 09:31",
                      "2026-06-14 09:32", "2026-06-14 09:33"]))

# Rééchantillonner : passer de la minute à la barre de 5 minutes
df["prix"].resample("5min").ohlc()  # open/high/low/close

# Moyenne mobile sur 3 périodes
df["prix"].rolling(3).mean()
```

Avec un index de dates, `resample("5min")` regroupe les données par tranches de 5 minutes — par exemple pour transformer des ticks en barres OHLC. `rolling(3).mean()` calcule une moyenne mobile sur 3 périodes, glissant le long de la série. Ce sont les opérations fondamentales pour construire des indicateurs.

Pourquoi ça compte. *Les données de marché sont par nature temporelles. Le rééchantillonnage et les fenêtres glissantes de Pandas permettent de passer des ticks aux barres et de calculer des indicateurs en quelques lignes — directement utile pour ton bot.*

2.6 Les dates et les formats de date

Les dates sont la principale source de problèmes lorsqu'on charge un fichier de marché : chaque source les écrit à sa façon, et un format mal interprété fausse tout le tri et toute l'analyse temporelle. Avant tout traitement chronologique, il faut **convertir les dates en vraies dates** — des objets `datetime` — et non les laisser sous forme de texte.

La solution : `pd.to_datetime`

```
df["date"] = pd.to_datetime(df["date"])  # devine les formats standards
df = pd.read_csv("ticks.csv", parse_dates=["date"])  # ou des la lecture
```


À la lecture d'un CSV, Pandas laisse souvent les dates comme du texte (object), ce qui fausse le tri. `pd.to_datetime` les convertit ; pour les formats standards (ISO), Pandas devine seul.

Dates mal écrites : préciser le format

```
df["date"] = pd.to_datetime(df["date"], format="%d/%m/%Y") # europeen
df["date"] = pd.to_datetime(df["date"], format="%Y-%m-%d %H:%M:%S") # avec heure
```

Préciser `format=` lève toute ambiguïté : « 03/04/2026 » devient sans équivoque le 3 avril (%d/%m/%Y) ou le 4 mars (%m/%d/%Y). Indispensable pour les dates européennes que Pandas interprète parfois à l'envers.

Dates invalides : errors=coerce

```
df["date"] = pd.to_datetime(df["date"], format="%d/%m/%Y", errors="coerce")
# -> les dates illisibles deviennent NaT, au lieu de planter
df = df.dropna(subset=["date"]) # on peut ensuite retirer ces lignes
```

Avec `errors="coerce"`, une date illisible n'interrompt pas le chargement : elle devient NaT (l'équivalent de NaN pour les dates), qu'on peut repérer puis écarter.

Date scindée en deux colonnes (en-tête décalé)

Cas piégeux : l'en-tête a **8 colonnes** (dont une « date »), mais les données comptent **9 colonnes**, car la date est écrite en deux morceaux (date et heure séparées par un espace pris pour un séparateur). Les colonnes se décalent.

```
# Solution 1 : imposer les 9 vrais noms, puis recombinaison la date
noms = ["date_j", "date_h", "symbole", "prix", "quantite",
        "cote", "volume", "ouverture", "cloture"]
df = pd.read_csv("ticks.csv", header=0, names=noms)
df["date"] = pd.to_datetime(df["date_j"] + " " + df["date_h"],
                           format="%Y-%m-%d %H:%M:%S")
df = df.drop(columns=["date_j", "date_h"])
```

```
# Solution 2 : si l'espace dans la date a été pris pour un séparateur,
#              forcer le bon séparateur de colonnes (la date reste entière)
df = pd.read_csv("ticks.csv", sep=",")
```

On impose les **vrais noms** (`names=`) pour réaligner, puis on **recolle** les morceaux de date par concaténation avant `to_datetime`. Le bon réflexe : inspecter le fichier brut (`df.head()`, `df.shape`) pour comprendre le décalage avant de corriger.

Extraire et reformater des composantes

```
df["date"].dt.year      # l'année
df["date"].dt.hour      # l'heure
df["date"].dt.day_name() # le jour de la semaine ("Monday"... )
df["date"].dt.strftime("%Y-%m-%d") # reformater en texte
```

Une fois converties, l'accesseur `.dt` donne accès aux composantes (utile pour filtrer sur les heures d'ouverture) et `strftime` reformate une date en texte (pour un rapport ou un nom de fichier).

Pourquoi ça compte. Sur un fichier de marché, la conversion des dates est presque toujours la première étape, et la plus piégeuse. Préciser `format=` pour les dates ambiguës, `errors="coerce"` pour ne pas planter, et inspecter le fichier brut quand le nombre de colonnes ne correspond pas à l'en-tête.

Les codes de format de date les plus courants :

Code	Signification (exemple : 14 juin 2026, 9h30)
%Y	Année sur 4 chiffres (2026).

Code	Signification (exemple : 14 juin 2026, 9h30)
%y	Annee sur 2 chiffres (26).
%m	Mois sur 2 chiffres (06).
%B	Nom de mois complet (juin).
%d	Jour sur 2 chiffres (14).
%H	Heure sur 24 h (09).
%M	Minutes (30).
%S	Secondes (00).
%d/%m/%Y	Format europeen : 14/06/2026.
%m/%d/%Y	Format americain : 06/14/2026.
%Y-%m-%d	Format ISO (recommande) : 2026-06-14.

Les opérations de date essentielles :

Opération	Ce qu'elle fait
<code>pd.to_datetime(col)</code>	Convertit du texte en vraies dates (devine les formats standards).
<code>pd.to_datetime(col, format="...")</code>	Convertit en precisant le format exact (dates ambiguës).
<code>errors="coerce"</code>	Met NaT sur les dates illisibles au lieu de planter.
<code>parse_dates=["col"]</code>	Convertit les dates des la lecture du CSV.
<code>pd.read_csv(..., names=[...])</code>	Impose les vrais noms (en-tete decale / date scindee).
<code>to_datetime(col_j + " " + col_h)</code>	Recombine une date scindee en deux colonnes.
<code>.dt.year / .dt.hour / ...</code>	Extrait une composante (annee, heure, jour...).
<code>.dt.day_name()</code>	Donne le jour de la semaine.
<code>.dt.strftime("...")</code>	Reformate une date en texte selon un motif.
<code>.dt.tz_localize("...")</code>	Attribue un fuseau (rend la date « avisee »).
<code>.dt.tz_convert("...")</code>	Convertit la colonne vers un autre fuseau.

3. La gestion du temps et des fuseaux horaires

Le temps est un sujet **piégeux** et critique en trading : les marchés sont dans des fuseaux horaires différents, et un horodatage mal géré peut fausser toute une analyse. Python fournit le module `datetime`, et la gestion des fuseaux est essentielle. Cette section approfondit ce que la section 2.6 a abordé côté Pandas, en travaillant sur des dates isolées.

3.1 Dates naïves et dates avisées

```
from datetime import datetime
from zoneinfo import ZoneInfo
```

```
# Une date SANS fuseau ("naive") : ambiguë, à éviter pour les marchés
t_naif = datetime(2026, 6, 14, 9, 30)
```

```
# Une date AVEC fuseau ("avisée") : sans ambiguïté
ny = datetime(2026, 6, 14, 9, 30, tzinfo=ZoneInfo("America/New_York"))
```

Une date **naïve** (sans fuseau) est ambiguë : 9h30, mais où ? Une date **avisée** (avec tzinfo) lève l'ambiguïté en désignant un instant précis. En trading, on travaille **toujours** avec des dates avisées : une heure sans fuseau ne veut rien dire quand on suit plusieurs marchés.

3.2 Convertir entre fuseaux

```
# Convertir un instant d'un fuseau à un autre (préserve le moment réel)
montreal = ny.astimezone(ZoneInfo("America/Montreal"))
londres = ny.astimezone(ZoneInfo("Europe/London"))
tokyo = ny.astimezone(ZoneInfo("Asia/Tokyo"))
```

```
# 9h30 à New York correspond à :
#   9h30 à Montréal (même fuseau)
#   14h30 à Londres
#   22h30 à Tokyo
```

astimezone convertit un instant d'un fuseau à l'autre en **préservant le moment réel** : c'est le même instant physique, exprimé dans un autre fuseau. New York et Montréal partagent le même fuseau (même heure), Londres est en avance de 5 heures, Tokyo de 13. Suivre des marchés à New York, Londres et Tokyo depuis Montréal exige ces conversions pour comparer les heures d'ouverture sans se tromper.

Pourquoi ça compte. Une erreur de fuseau peut décaler des données de plusieurs heures et fausser des signaux ou des réconciliations. C'est l'une des erreurs les plus coûteuses et les plus discrètes dans un système connecté à des marchés mondiaux. Le réflexe : tout stocker en avisé, idéalement en UTC, et ne convertir que pour l'affichage.

3.3 Construire et formater des dates

```
from datetime import datetime, timedelta

# Lire une date depuis du texte (avec le format)
ouverture = datetime.strptime("2026-06-14 09:30", "%Y-%m-%d %H:%M")

# Ecrire une date en texte (formater)
ouverture.strftime("%d/%m/%Y")      # -> "14/06/2026"

# Maintenant, et calculs de durée
maintenant = datetime.now(ZoneInfo("America/Montreal"))
dans_une_heure = maintenant + timedelta(hours=1)
duree = dans_une_heure - maintenant    # timedelta de 1 heure
```

Deux opérations symétriques : strptime lit une date depuis du texte (le « p » comme *parse*), strftime l'écrit en texte (le « f » comme *format*). Les codes de format sont les mêmes qu'en section 2.6. timedelta représente une durée : on l'ajoute ou la soustrait à une date pour calculer des échéances, et la différence entre deux dates donne un timedelta (par exemple pour mesurer une latence).

3.4 Opérations de temps essentielles

Opération	Ce qu'elle fait
<code>datetime(2026, 6, 14, 9, 30)</code>	Cree une date (naive, sans fuseau).
<code>tzinfo=ZoneInfo("...")</code>	Attribue un fuseau (rend la date avisee).
<code>dt.astimezone(ZoneInfo("..."))</code>	Convertit vers un autre fuseau (meme instant reel).
<code>datetime.strptime(txt, "...")</code>	Lit une date depuis du texte selon un format.
<code>dt.strftime("...")</code>	Ecrit une date en texte selon un format.
<code>datetime.now(ZoneInfo("..."))</code>	L'instant present dans un fuseau donne.
<code>timedelta(hours=1, minutes=30)</code>	Une duree, a ajouter ou soustraire a une date.
<code>date2 - date1</code>	Difference entre deux dates (un timedelta).

Pourquoi ça compte. Une erreur de fuseau horaire peut décaler des données de plusieurs heures et fausser des signaux ou des réconciliations. Manipuler des horodatages avisés (avec fuseau) est une discipline indispensable dans un système connecté à des marchés mondiaux.

4. Comprendre les paramètres clés

Tout au long de ce document reviennent des **paramètres** qu'on retrouve dans de nombreuses fonctions : `axis`, `format`, `errors`, `parse_dates`, `tzinfo`, etc. Cette section les explique. Pour chacun, on précise à quoi il se **rattache** — une bibliothèque (NumPy, Pandas), un type d'objet (date `datetime`), ou une fonction précise — car un même nom peut exister ailleurs avec un sens différent, et savoir d'où il vient aide à le retenir.

Pourquoi ça compte. Comment lire les étiquettes. Chaque paramètre est marqué de son rattachement entre crochets : `[Pandas + NumPy]` s'il est partagé par les deux, `[Pandas — lecture CSV]` s'il est propre à `read_csv`, `[datetime]` s'il appartient au module de dates standard, etc. Un paramètre peut être très général (`axis`) ou spécifique à une seule fonction (`parse_dates`).

axis — la direction d'une opération `[Pandas + NumPy, très général]`

```
# axis=0 : verticalement (sur les lignes, colonne par colonne) -> DEFAULT
# axis=1 : horizontalement (sur les colonnes, ligne par ligne)
df.sum(axis=0)           # somme de chaque colonne (par default)
df.sum(axis=1)           # somme de chaque ligne
(df[cols] > 0).all(axis=1) # toutes les colonnes OK, ligne par ligne
```

`axis` indique la **direction** d'une opération. `axis=0` (défaut) travaille verticalement, le long des lignes — donc « pour chaque colonne ». `axis=1` travaille horizontalement — donc « pour chaque ligne ».

Rattachement : c'est l'un des paramètres les plus universels — on le retrouve dans presque toutes les opérations d'agrégation de NumPy et de Pandas, toujours avec le même sens. Retiens `axis=1` = « ligne par ligne ».

format — le motif d'une date `[datetime + Pandas (to_datetime)]`

```
pd.to_datetime(col, format="%d/%m/%Y") # Pandas : toute une colonne
datetime.strptime(txt, "%Y-%m-%d %H:%M") # datetime : une date isolée
```

format décrit, avec des codes (%d jour, %m mois, %Y année...), comment une date est écrite, pour lever les ambiguïtés. **Rattachement** : propre à la manipulation de dates. On le passe à `pd.to_datetime` (Pandas, sur une colonne) comme à `datetime.strptime / strftime` (module `datetime`, sur une date isolée) — mêmes codes de format dans les deux cas.

errors — comment réagir à une valeur invalide [Pandas (to_numeric, to_datetime)]

```
pd.to_numeric(col, errors="coerce") # valeur illisible -> NaN
pd.to_datetime(col, errors="coerce") # date illisible -> NaT
# errors="raise" (default) : lève une erreur et arrête tout
# errors="ignore" : laisse la valeur telle quelle
```

errors décide du comportement face à une valeur qu'on n'arrive pas à convertir : "raise" (défaut) lève une erreur ; "coerce" remplace la valeur fautive par NaN / NaT et continue. **Rattachement** : spécifique aux fonctions de conversion de Pandas, surtout to_numeric et to_datetime. On ne le retrouve pas sur les opérations de calcul.

parse_dates, index_col, sep, names — la lecture d'un CSV [Pandas (read_csv)]

```
df = pd.read_csv("ticks.csv",
                  parse_dates=["date"], # convertir cette colonne en dates
                  index_col="date",    # et en faire l'index
                  sep=";",              # le séparateur de colonnes
                  names=[...])         # imposer les noms (si pas d'en-tête)
```

Ces quatre paramètres pilotent le chargement d'un fichier : parse_dates convertit des colonnes en dates dès la lecture ; index_col désigne la colonne qui devient l'index ; sep précise le séparateur de colonnes ; names impose les noms (en-tête absent ou décalé). **Rattachement** : ils sont **propres à read_csv** (et à ses cousins read_parquet, read_excel pour certains). Inutile de les chercher ailleurs : ce sont des options de lecture de fichier.

Pourquoi parse_dates=["date"] est plus sûr ?

Sans parse_dates, pandas lit la colonne date comme du texte (object). Tu obtiens des chaînes "1982-06-12", et il faut te souvenir de les convertir ensuite avec pd.to_datetime(...). Trois risques :

1. L'étape oubliée — tu connais le piège récurrent : une conversion non réaffectée (pd.to_datetime(...)) jeté) ou simplement oubliée → tu te retrouves avec du texte. parse_dates fait la conversion au chargement, donc une étape de moins à rater.
2. Tri/calculs faux — sur du texte, sort_values("date") trie alphabétiquement, pas chronologiquement (l'ISO s'en sort par chance, mais dès qu'un format est moins propre, ça casse), et tu ne peux pas calculer d'écarts entre dates.
3. Garantie de type — juste après read_csv, tu sais que date est datetime64. Le contrat est explicite, pas « à vérifier plus tard ».

tzinfo — le fuseau horaire d'une date [datetime (objet date isolée)]

```
from zoneinfo import ZoneInfo
datetime(2026, 6, 14, 9, 30, tzinfo=ZoneInfo("America/New_York"))
```

tzinfo attache un **fuseau horaire** à une date, la rendant « avisée » (sans ambiguïté). **Rattachement** : c'est un paramètre du constructeur datetime du module standard, donc pour une **date isolée**. Pour faire la même chose sur une colonne entière de DataFrame, ce n'est PAS tzinfo mais les méthodes Pandas dt.tz_localize / dt.tz_convert. Même idée, outil différent selon qu'on manipule une date ou une colonne.

ascending, how, subset — d'autres paramètres Pandas fréquents [Pandas (fonctions précises)]

```
df.sort_values("prix", ascending=False) # sort_values : sens du tri
pd.merge(a, b, on="symbole", how="inner") # merge : type de jointure
df.dropna(subset=["date"])                # dropna : sur quelles colonnes
```

ascending (sens du tri, croissant par défaut) appartient à sort_values ; how (type de jointure : "inner", "outer", "left") à merge ; subset (restreindre à certaines colonnes) à dropna et drop_duplicates. **Rattachement** : chacun est lié à une fonction Pandas précise — contrairement à axis, ils ne sont pas universels.

result_type — la forme du résultat de apply [Pandas (apply)]

```
df[cols] = df[cols].apply(lambda L: sorted(L),
                        axis=1, result_type="expand")
```

result_type="expand" demande à apply d'étaler une liste de résultats sur plusieurs colonnes (au lieu de la garder dans une seule). **Rattachement** : très **spécifique** — propre à la méthode apply de Pandas. C'est l'exemple type du paramètre qu'on ne croise qu'à un seul endroit, mais qu'il faut connaître pour le cas du tri intra-ligne.

inplace — modifier sur place ou renvoyer une copie [Pandas (à éviter)]

```
df.dropna(inplace=True)      # modifie df directement (déconseille)
df = df.dropna()             # renvoie une copie modifiée (recommande)
```

inplace=True modifie l'objet directement au lieu d'en renvoyer une copie. **Rattachement** : présent sur beaucoup de méthodes Pandas, mais la pratique moderne recommande de **ne pas l'utiliser** et de réassigner (df = df.dropna()) pour un code plus clair et sans surprise.

Les paramètres essentiels en un coup d'œil :

Paramètre	Rattachement	Rôle
axis	Pandas + NumPy	Direction : 0 = par colonne (défaut), 1 = par ligne.
format	datetime + to_datetime	Motif exact d'une date, pour lever les ambiguïtés.
errors	Pandas (to_numeric / to_datetime)	"coerce" met NaN/NaT sur les valeurs invalides.
parse_dates	Pandas (read_csv)	Convertit des colonnes en dates dès la lecture.
index_col	Pandas (read_csv)	Colonne qui devient l'index du DataFrame.
sep	Pandas (read_csv)	Séparateur de colonnes (virgule, ; , tabulation).
names	Pandas (read_csv)	Impose les noms de colonnes (en-tête absent/décalé).
tzinfo	datetime (date isolée)	Attache un fuseau à une date (la rend « avisée »).
ascending	Pandas (sort_values)	Sens du tri (croissant par défaut).
how	Pandas (merge)	Type de jointure : inner / outer / left.
subset	Pandas (dropna, drop_duplicates)	Restreint l'opération à certaines colonnes.
result_type	Pandas (apply)	"expand" étale le résultat sur plusieurs colonnes.
inplace	Pandas (la plupart)	Modifie sur place ; préférer réassigner.

Pourquoi ça compte. Quelques paramètres reviennent partout (axis), d'autres sont propres à une seule fonction (parse_dates, result_type). Savoir auquel chaque paramètre se rattache évite deux pièges : le chercher là où il n'existe pas, et oublier qu'une valeur par défaut (axis=0, ascending=True, errors="raise") gouverne déjà le comportement quand on ne précise rien.

Récapitulatif

- 1. NumPy** : la vectorisation applique une opération à tout un tableau d'un coup, en code C rapide ; éviter les boucles Python. Le code natif relâche le GIL et peut, pour les calculs lourds, utiliser plusieurs cœurs.
- 2. Pandas DataFrame** : un tableau à colonnes nommées ; charger (`read_csv`), nettoyer (types, NaN, renommer), filtrer, grouper, trier, fusionner en quelques lignes.
- 3. Préparation des données** : corriger les types (`astype`, `to_numeric`), combler les trous (`fillna`, `ffill`), normaliser (min-max, score z), et surtout convertir les dates (`to_datetime`, `format=`, `errors=coerce`).
- 4. Séries temporelles** : indexer par date, rééchantillonner (`resample`), fenêtres glissantes (`rolling`).
- 5. Le temps** : toujours des dates avisées (avec fuseau) ; convertir avec `astimezone` (date isolée) ou `tz_convert` (colonne) pour des marchés mondiaux.
- 6. Les paramètres clés** : `axis` (0 colonne / 1 ligne), `format` (motif de date), `errors="coerce"` (ne pas planter), `parse_dates` et `sep` (lecture CSV), `tzinfo` (fuseau). Ils reviennent partout et ont des valeurs par défaut.

Questions de vérification

Essaie de répondre avant de lire la réponse, fournie juste en dessous, en retrait.

Sur NumPy

Q1. Qu'est-ce que la vectorisation et pourquoi est-elle rapide ?

Réponse. Appliquer une opération à tout un tableau d'un coup, sans boucle Python. C'est rapide parce que l'opération s'exécute en code C optimisé sous le capot, bien plus vite qu'une boucle Python.

Q2. La vectorisation libère-t-elle le GIL, et est-ce la raison de sa vitesse ?

Réponse. Le code natif appelé par NumPy relâche effectivement le GIL, ce qui permet aux opérations lourdes (comme `A @ B`) d'utiliser plusieurs cœurs. Mais la source principale du gain est ailleurs : éviter l'interpréteur Python (code C, même sur un seul cœur). Les deux effets se cumulent, sans être identiques.

Q3. Que fait `prix[prix > 5232]` ?

Réponse. C'est un filtrage booléen : il renvoie tous les éléments du tableau supérieurs à 5232, sans boucle ni condition explicite.

Sur Pandas

Q4. Qu'est-ce qu'un DataFrame, et comment en charge-t-on un depuis un fichier ?

Réponse. Un tableau à colonnes nommées, comme une feuille de calcul. On le charge avec `pd.read_csv` (ou `pd.read_parquet`), éventuellement avec `parse_dates` et `index_col` pour un index temporel.

Q5. Comment change-t-on le type d'une colonne, et que faire si certaines valeurs sont illisibles ?

Réponse. Avec `astype` (`int`, `float`, `str`, `bool`). Si des valeurs ne sont pas convertibles, on utilise `pd.to_numeric(col, errors="coerce")`, qui met NaN sur les valeurs illisibles au lieu de planter.

Q6. Comment tester une condition sur plusieurs colonnes, ligne par ligne ?

Réponse. On sélectionne les colonnes (`df[cols]`), on applique la condition, puis `.all(axis=1)` vérifie que toutes la satisfont sur chaque ligne (ou `.any(axis=1)` pour au moins une). `axis=1` signifie « horizontalement ».

Q7. À quoi sert `groupby` ?

Réponse. À regrouper les lignes par valeur d'une colonne puis appliquer un calcul (somme, moyenne...) à chaque groupe. C'est l'équivalent d'un tableau croisé dynamique.

Q8. À quoi servent `resample` et `rolling` ?

Réponse. `resample` change la fréquence d'une série temporelle (par exemple regrouper des minutes en barres de 5 minutes). `rolling` calcule sur une fenêtre glissante (par exemple une moyenne mobile).

Sur les dates et le temps

Q9. Pourquoi convertir les dates avec `to_datetime`, et que fait `errors="coerce"` ?

Réponse. Parce que les dates lues d'un CSV sont souvent du texte, ce qui fausse le tri. `to_datetime` les convertit en vraies dates. `errors="coerce"` remplace les dates illisibles par `NaT` au lieu de faire planter le chargement.

Q10. Une date est écrite « 03/04/2026 ». Comment lever l'ambiguïté ?

Réponse. En précisant le format : `pd.to_datetime(col, format="%d/%m/%Y")` pour le 3 avril, ou `format="%m/%d/%Y"` pour le 4 mars. Sans format explicite, Pandas peut se tromper sur les dates européennes.

Q11. Quelle est la différence entre une date naïve et une date avisée, et comment convertit-on entre fuseaux ?

Réponse. Une date naïve n'a pas de fuseau (ambiguë) ; une date avisée inclut un fuseau (`tzinfo`), donc un instant précis. On convertit avec `astimezone` (date isolée) ou `.dt.tz_convert` (colonne entière), ce qui préserve le moment réel.

Sur les paramètres

Q12. Que signifie `axis=1` dans une opération Pandas ?

Réponse. Que l'opération se fait horizontalement, ligne par ligne (sur les colonnes de chaque ligne). `axis=0`, la valeur par défaut, travaille verticalement, colonne par colonne. C'est le paramètre clé pour tester ou trier les valeurs d'une même ligne.

Q13. À quoi sert `errors="coerce"`, et sur quelles fonctions le retrouve-t-on ?

Réponse. À remplacer les valeurs invalides par `NaN` (ou `NaT` pour une date) au lieu de faire planter le programme. On le retrouve sur `pd.to_numeric` et `pd.to_datetime`, notamment pour charger des fichiers aux données imparfaites.

Q14. Quelle est la différence entre un paramètre positionnel et un paramètre nommé ?

Réponse. Un paramètre positionnel est reconnu par sa position (le 1er, le 2e...). Un paramètre nommé est passé sous la forme `nom=valeur`, ce qui le rend explicite et permet de ne fournir que ceux qu'on veut ; les autres gardent leur valeur par défaut.

Document suivant (6/8) : l'interaction avec le monde extérieur — requêtes HTTP, WebSockets pour le temps réel, JSON, bases de données et fichiers.